

Faculdade de Computação e Informática – FCI

Curso: Ciência da Computação

Laboratório de Computação Paralela - Turma 05N-11

Prof. Mario Menezes

Laboratório de PThreads

Programação Concorrente em Memória

Compartilhada e Análise do GIL

Nome: Gustavo Henrique de Sousa Santos

RA: 10721355

[Clique aqui: Link para o repositório GitHub](#)

Data de Submissão: 20/09/2026

Sumário

1. Introdução e Fundamentos Teóricos.....	3
2. Metodologia e Ambiente Experimental.....	3
2.1. Ambiente de Hardware e Software.....	3
2.2. Ferramentas e Configuração de Compilação (Parte 1 e 2).....	4
2.3. Execução e Análise de Desempenho (Python GIL - Parte 3).....	4
2.4. Metodologia de Coleta e Avaliação de Dado.....	4
3. Evidências Experimentais (Capturas de Tela Obrigatórias).....	4
3.1. Parte 1: O Problema da Conta Corrente.....	4
3.2. Parte 2: Menu de Problemas (Escolhidos: B, C, E).....	5
3.3. Parte 3: O Desafio do Python e o GIL (Global Interpreter Lock).....	6
4. Tabela Comparativa de Desempenho e Gráficos de Speedup.....	6
4.1. Tabela com tempos médios de execução de todas as etapas práticas (exceto da parte 2, vide seção 4.2).....	6
4.2. Tabela de escalabilidade para as 3 opções do Menu.....	7
4.3. Gráficos de Speedup × Número de Threads e análise discursiva.....	8
5. Respostas às 9 Questões de Reflexão.....	9
5.1. Ciclo de Leitura-Modificação-Escrita (Parte 1 - Fase 1).....	9
5.2. Impacto e Granularidade do Mutex (Parte 1 - Fase 2).....	10
5.3. Estratégias Alternativas de Sincronização (Parte 1).....	10
5.4. Decomposição e Balanceamento de Carga (Parte 2).....	10
5.5. Reentrância e Thread-Safety em Bibliotecas (Parte 2 - Opção B).....	11
5.6. Escalabilidade e Limitações Físicas de Memória (Parte 2 - Opção C/E).....	11
5.7. O Papel do GIL no CPython (Parte 3).....	11
5.8. Paralelismo Real em Python (Parte 3).....	12
5.9. Limites Teóricos e Lei de Amdahl.....	12
6. Seção de Dificuldades Técnicas e Soluções.....	12
7. Seção Obrigatória: Declaração e Análise do Uso de IA.....	13
7.1. Registro de Uso de Ferramentas.....	13
7.2. Listagem das Principais Solicitações (Prompts).....	13
7.3. Análise Crítica Obrigatória.....	14
8. Conclusão e Referências.....	14
Referências Bibliográficas:.....	14

1. Introdução e Fundamentos Teóricos

A programação paralela em arquiteturas de memória compartilhada permite que múltiplos fluxos de execução operem simultaneamente sobre o mesmo espaço de endereçamento. Neste paradigma, um único processo pode instanciar várias threads, que atuam como instâncias de execução "leves" em comparação a processos tradicionais. O padrão POSIX Threads (Pthreads), implementado amplamente em sistemas baseados em Unix, fornece a API (Application Programming Interface) padrão em linguagem C para o gerenciamento e a sincronização desses fluxos concorrentes.

O ciclo de vida básico de uma aplicação multithread envolve a criação dos fluxos através da função `pthread_create` e a sincronização final com a rotina principal utilizando `pthread_join`, que obriga o processo a aguardar a conclusão do trabalho das threads. Embora esse modelo permita alto desempenho e redução do tempo de execução através da divisão de trabalho, ele introduz desafios críticos de concorrência. Quando múltiplas threads tentam acessar e modificar variáveis globais compartilhadas de forma simultânea, podem ocorrer condições de corrida (*race conditions*), resultando em perda de atualizações e inconsistência de dados.

Para prevenir a corrupção de variáveis, torna-se imprescindível a utilização de mecanismos de exclusão mútua. Estruturas como o mutex (via `pthread_mutex_t`) delimitam seções críticas do código, garantindo que apenas uma única thread acesse e modifique o recurso protegido por vez. Contudo, o uso de bloqueios de granularidade muito fina pode introduzir uma sobrecarga (overhead) substancial ao sistema, gerando alta contenção e neutralizando os ganhos do paralelismo. Outro aspecto fundamental para a confiabilidade de sistemas concorrentes é a exigência de funções thread-safe ou reentrantes; funções nativas da linguagem C (como `rand`) que armazenam estado global podem causar comportamentos imprevisíveis ao serem acessadas por várias threads, exigindo versões específicas (re-entrantes) adequadas para concorrência.

Por fim, o estudo de concorrência estende-se para além do C, abrangendo as restrições arquiteturais de linguagens modernas. No interpretador padrão do Python (*CPython*), o mecanismo denominado *Global Interpreter Lock* (GIL) atua como um mutex em nível de máquina virtual, impedindo a execução de múltiplos bytecodes simultâneos na mesma instância. Essa restrição torna a biblioteca nativa de threads do Python ineficaz para obter aceleração real em algoritmos com limitação imposta por processamento de CPU (CPU-bound), exigindo alternativas de mais alto custo computacional, como a comunicação via múltiplos processos isolados.

2. Metodologia e Ambiente Experimental

2.1. Ambiente de Hardware e Software

Os experimentos práticos e a compilação dos códigos em linguagem C foram conduzidos integralmente em um ambiente de desenvolvimento isolado fornecido pelo GitHub Codespaces. Esse ambiente em nuvem operava sobre uma infraestrutura baseada na distribuição Ubuntu 24.04.5 LTS (codinome Noble Numbat), executando a versão de kernel Linux 6.8.0-1064-azure.

A máquina virtual alocada para os testes contava com um processador AMD EPYC 7763 64-Core Processor, operando com uma frequência base aproximada de 2.45 GHz. A arquitetura de hardware disponibilizou 1 núcleo físico, operando com 2

threads lógicas. O sistema de memória cache possuía L1d e L1i com 32 KiB cada, L2 de 512 KiB e L3 compartilhada de 32 MiB. A capacidade de memória RAM disponível para a instância era de 7.8 GiB.

2.2. Ferramentas e Configuração de Compilação (Parte 1 e 2)

Os problemas de simulação bancária e os algoritmos numéricos (Monte Carlo, Soma de Vetores e Matriz-Vetor) foram implementados em linguagem C. Utilizou-se o compilador GCC versão 13.3.0 em conjunto com a biblioteca padrão GLIBC versão 2.39. A seguinte flag de compilação padrão foi utilizada para garantir otimização e a ativação correta da biblioteca POSIX Threads: `gcc -O2 -Wall nome_do_arquivo.c -o nome_do_executavel -pthread`

2.3. Execução e Análise de Desempenho (Python GIL - Parte 3)

Para a verificação do impacto do *Global Interpreter Lock* (GIL) em tarefas de alta carga computacional, utilizou-se o interpretador *CPython* padrão na versão 3.14.2. Foram desenvolvidos dois scripts (`seq.py` e `threads.py`) comparando a execução sequencial contra a concorrente baseada no módulo nativo `threading`. Os tempos de execução foram mensurados internamente utilizando a biblioteca `time` do próprio Python.

2.4. Metodologia de Coleta e Avaliação de Dado

Na avaliação de escalabilidade (Parte 2), os códigos foram executados variando a quantidade de thread 1, 2, 4 e 8. O tempo de execução reportado foi coletado via função de alta precisão `clock_gettime(CLOCK_MONOTONIC)`. A partir da obtenção do tempo base sequencial com 1 thread (T_1) e dos tempos paralelos (T_p), calculou-se a curva de aceleração ($SpeedUp(S_p) = T_1 / T_p$) e a eficiência ($E_p = S_p / p$) do paralelismo aplicado.

3. Evidências Experimentais (Capturas de Tela Obrigatórias)

3.1. Parte 1: O Problema da Conta Corrente

Fase 0: Sequencial:

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ gcc -O2 -Wall contacorrente_seq.c -o contacorrente_seq -pthread
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.0951 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.1042 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.1008 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.1033 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.0992 s
```

Fase 1: Condição de Corrida (Race Condition):

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ gcc -O2 -Wall contacorrente_f1.c -o contacorrente_f1 -pthread
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0493 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: -99999000.00 | Tempo: 0.0499 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: -99999000.00 | Tempo: 0.0490 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0498 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0494 s
```

Fase 2: Correção com Mutex:

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ gcc -O2 -Wall contacorrente_f2.c -o contacorrente_f2 -pthread
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 4.1696 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 4.0048 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 3.4990 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 3.5486 s
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 3.1088 s
```

3.2. Parte 2: Menu de Problemas (Escolhidos: B, C, E)

Opção B: Cálculo de π pelo Método de Monte Carlo:

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ gcc -O2 -Wall monte_carlo_pi.c -o monte_carlo_pi -pthread
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./monte_carlo_pi 100000000 1
./monte_carlo_pi 100000000 2
./monte_carlo_pi 100000000 4
./monte_carlo_pi 100000000 8
[Monte Carlo Pi] N=100000000 | Threads=1 | Pi: 3.141554 | Tempo: 1.2419 s
[Monte Carlo Pi] N=100000000 | Threads=2 | Pi: 3.141614 | Tempo: 0.8435 s
[Monte Carlo Pi] N=100000000 | Threads=4 | Pi: 3.141662 | Tempo: 0.7594 s
[Monte Carlo Pi] N=100000000 | Threads=8 | Pi: 3.141496 | Tempo: 0.7825 s
```

Opção C: Soma de Vetores Gigantes:

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ gcc -O2 -Wall soma_vetores.c -o soma_vetores -pthread
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./soma_vetores 50000000 1
./soma_vetores 50000000 2
./soma_vetores 50000000 4
./soma_vetores 50000000 8
[Soma Vetores] N=50000000 | Threads=1 | Tempo: 0.2741 s
[Soma Vetores] N=50000000 | Threads=2 | Tempo: 0.1581 s
[Soma Vetores] N=50000000 | Threads=4 | Tempo: 0.1723 s
[Soma Vetores] N=50000000 | Threads=8 | Tempo: 0.1672 s
```

Opção E: Multiplicação de Matriz por Vetor ($M \times V$):

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ gcc -O2 -Wall matriz_vetor.c -o matriz_vetor -pthread
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ ./matriz_vetor 10000 1
./matriz_vetor 10000 2
./matriz_vetor 10000 4
./matriz_vetor 10000 8
[Matriz-Vetor] N=10000 | Threads=1 | Tempo: 0.0971 s
[Matriz-Vetor] N=10000 | Threads=2 | Tempo: 0.0521 s
[Matriz-Vetor] N=10000 | Threads=4 | Tempo: 0.0539 s
[Matriz-Vetor] N=10000 | Threads=8 | Tempo: 0.0526 s
```

3.3. Parte 3: O Desafio do Python e o GIL (Global Interpreter Lock)

Código 1: Sequencia e Código 2: Concorrente com Threads!:

```
• @Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ python3 seq.py
python3 threads.py
[Python Sequencial] Tempo: 5.9539 segundos
[Python com Threads] Tempo: 5.0837 segundos
• @Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ python3 seq.py
python3 threads.py
[Python Sequencial] Tempo: 5.4457 segundos
[Python com Threads] Tempo: 5.1298 segundos
• @Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ python3 seq.py
python3 threads.py
[Python Sequencial] Tempo: 5.6137 segundos
[Python com Threads] Tempo: 4.7735 segundos
• @Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ python3 seq.py
python3 threads.py
[Python Sequencial] Tempo: 5.4170 segundos
[Python com Threads] Tempo: 4.8296 segundos
• @Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_PThreads (main) $ python3 seq.py
python3 threads.py
[Python Sequencial] Tempo: 5.4332 segundos
[Python com Threads] Tempo: 5.0779 segundos
```

4. Tabela Comparativa de Desempenho e Gráficos de Speedup

4.1. Tabela com tempos médios de execução de todas as etapas práticas (exceto da parte 2, vide seção 4.2)

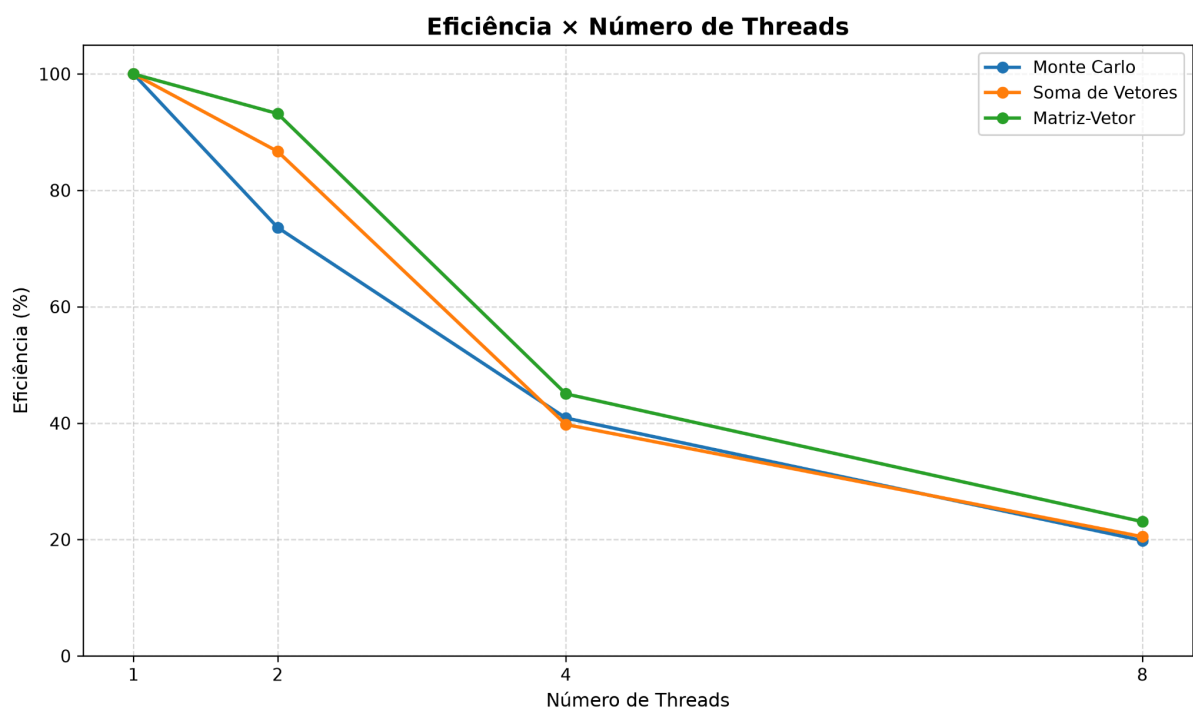
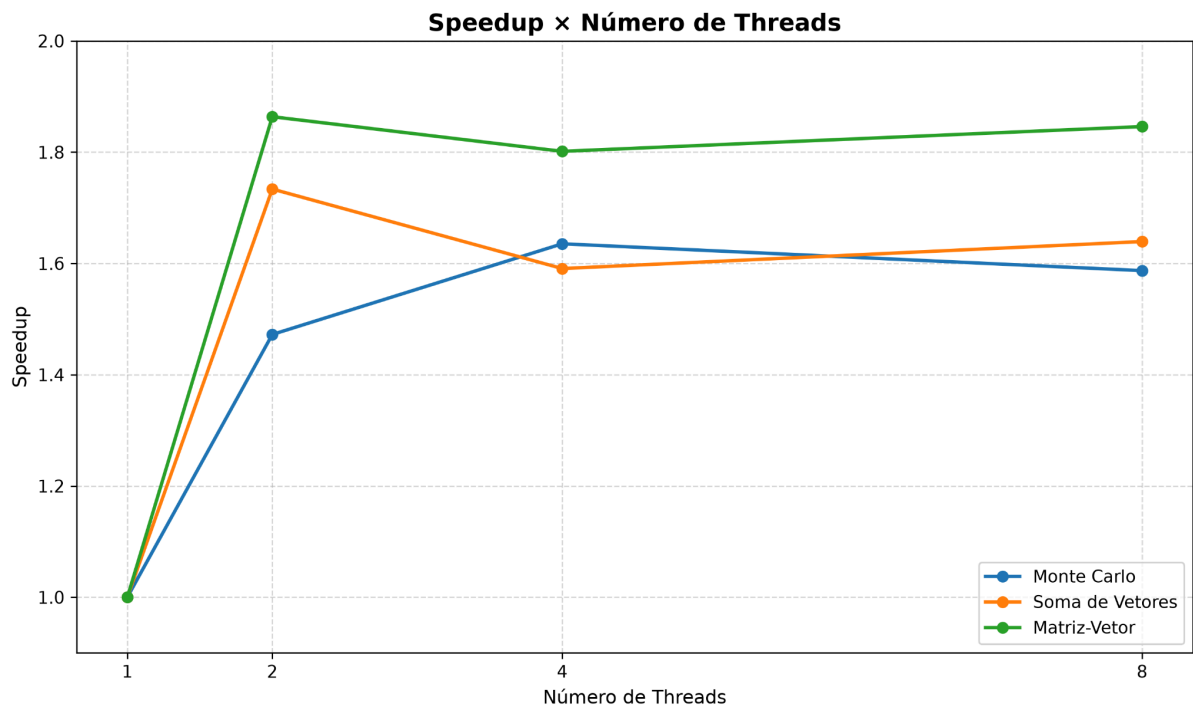
Opção do Menu	Execução	Tempo da execução (s)	Tempo Médio (s)
Conta Corrente Sequencial	1	0,0951	0,1005
	2	0,1042	
	3	0,1008	
	4	0,1033	
	5	0,0992	
Conta Corrente Race Condition	1	0,0493	0,0495
	2	0,0499	
	3	0,0490	
	4	0,0498	
	5	0,0494	
Conta Corrente Mutex	1	4,1696	3,6662
	2	4,0048	
	3	3,4990	

	4	3,5486	
	5	3,1088	
Python Sequencial	1	5,9539	5,5727
	2	5,4457	
	3	5,6137	
	4	5,4170	
	5	5,4332	
Python Concorrente	1	5,0837	4,9789
	2	5,1298	
	3	4,7735	
	4	4,8296	
	5	5,0779	

4.2. Tabela de escalabilidade para as 3 opções do Menu

Opção do Menu	Threads (p)	Tempo Médio (Tp)	Speedup (Sp)	Eficiência (Ep)
Monte Carlo	1	1,2419	1,0000	100%
	2	0,8435	1,4723	74%
	4	0,7594	1,6354	41%
	8	0,7825	1,5871	20%
Soma de Vetores	1	0,2741	1,0000	100%
	2	0,1581	1,7337	87%
	4	0,1723	1,5908	40%
	8	0,1672	1,6394	20%
Matriz-Vetor	1	0,0971	1,0000	100%
	2	0,0521	1,8637	93%
	4	0,0539	1,8015	45%
	8	0,0526	1,8460	23%

4.3. Gráficos de Speedup × Número de Threads e análise discursiva



A análise dos resultados obtidos através da experimentação prática com 1, 2, 4 e 8 threads evidencia um comportamento clássico de escalabilidade em arquiteturas de memória compartilhada: o ganho de desempenho inicial seguido de uma rápida saturação da aceleração (Speedup).

Comportamento com 2 Threads: Em todos os três problemas avaliados (Monte Carlo, Soma de Vetores e Multiplicação Matriz-Vetor), a duplicação dos fluxos de execução reduziu o tempo de processamento de forma expressiva, refletindo em altos índices de eficiência (74% para Monte Carlo, 87% para Soma de Vetores e 93% para Matriz-Vetor). Isso ocorre

porque os núcleos físicos disponíveis conseguiram absorver a carga dividida sem contenção acentuada de hardware.

Saturação e Limitação de Desempenho (4 e 8 Threads): Ao elevar o número de threads para 4 e 8, o Speedup estabiliza em torno de 1.6x a 1.8x, enquanto a eficiência cai acentuadamente (chegando a 20% com 8 threads). Esse fenômeno decorre de dois fatores principais: a sobrecarga de criação e troca de contexto e o gargalo de largura de banda de memória.

O custo computacional associado às chamadas de sistema para criar e sincronizar as threads (*pthread_create* e *pthread_join*) passa a consumir uma fração relevante do tempo total de execução, visto que o tamanho do problema é pulverizado rapidamente. Além disso, particularmente na Soma de Vetores e na Matriz-Vetor, as operações aritméticas elementares executadas pela CPU são extremamente rápidas. O fator limitante absoluto torna-se a taxa de transferência do barramento da memória RAM. Com múltiplas threads requisitando dados simultaneamente para as caches, o barramento atinge a sua saturação máxima, fazendo com que os núcleos fiquem ociosos aguardando a chegada dos dados da memória principal, o que impede qualquer ganho adicional de aceleração.

5. Respostas às 9 Questões de Reflexão

5.1. Ciclo de Leitura-Modificação-Escrita (Parte 1 - Fase 1)

A instrução `saldo += VALOR_DEPOSITO` no código em C parece única, mas em nível de instruções de máquina ela não é atômica. A Unidade Central de Processamento (CPU) executa esse comando em três ciclos principais de hardware distintos:

Leitura (Fetch): O valor atual da variável compartilhada `saldo` é copiado da memória RAM (ou da cache L1/L2) para um registrador da CPU.

Modificação: A Unidade Lógica Aritmética (ULA) soma o valor do depósito ao valor que está no registrador.

Escrita (Write): O resultado atualizado do registrador é gravado de volta na memória RAM.

Na Fase 1, ocorre a Perda de Atualizações (Lost Updates) porque as threads se sobrepõem no tempo. Uma thread de depósito pode ler o saldo simultaneamente a uma thread de saque. Ambas realizam a matemática em seus próprios registradores baseadas no mesmo saldo desatualizado e, ao escreverem o resultado de volta na RAM, uma sobrescreve e anula completamente a operação da outra.

5.2. Impacto e Granularidade do Mutex (Parte 1 - Fase 2)

A estrutura *pthread_mutex_t* resolve essa inconsistência matemática ao delimitar o que chamamos de Seção Crítica. O mutex age como um bloqueio de exclusão mútua: se a thread de saque adquirir o bloqueio (*Lock*), a thread de depósito é forçada pelo sistema operacional a suspender sua execução e aguardar até que a seção crítica seja liberada.

Apesar de garantir a consistência, o impacto temporal na Fase 2 é severo devido à granularidade fina do bloqueio. Como a aquisição (*lock*) e liberação (*unlock*) ocorrem dentro de um laço repetido 100 milhões de vezes (somando depósitos e saques), o programa sofre um imenso overhead de sincronização. O tempo gasto pelo sistema operacional realizando trocas de contexto e gerenciando a contenção das threads se torna infinitamente maior do que o tempo gasto fazendo a soma aritmética em si.

5.3. Estratégias Alternativas de Sincronização (Parte 1)

Para resolver o problema da conta corrente mantendo a precisão sem sofrer o overhead absurdo de 100 milhões de chamadas de lock, pode-se adotar duas estratégias conceituais alternativas:

(a) Variáveis locais por thread com redução final: Ao invés de acessar a variável global saldo a cada iteração, cada thread declara uma variável local (por exemplo, subtotal). As threads realizam o laço de 50 milhões de iterações alterando apenas sua variável local (que fica isolada em seus próprios registradores). Ao final do laço, utiliza-se um único *pthread_mutex_lock* apenas para somar o subtotal daquela thread ao saldo global. Isso reduz as operações de lock de 100 milhões para apenas 2.

(b) Utilizando operações atômicas nativas: Utilizando a biblioteca padrão *<stdatomic.h>* do C11, pode-se declarar a variável como *_Atomic double saldo*. O compilador traduzirá a soma para instruções específicas de assembly (como o prefixo LOCK em arquiteturas x86) que garantem a atomicidade diretamente no barramento de memória em nível de hardware, dispensando a criação de estruturas de mutex controladas pelo software.

5.4. Decomposição e Balanceamento de Carga (Parte 2)

Estratégia de particionamento: Nos problemas implementados da Parte 2 (como Monte Carlo, Soma de Vetores e Multiplicação Matriz-Vetor), foi utilizada a estratégia de particionamento estático por blocos contíguos. O espaço total de trabalho (índices N ou linhas da matriz) é dividido igualmente entre o número total de threads ($N / \text{num_threads}$). Cada thread calcula seu próprio intervalo de atuação independente (início e fim), garantindo localidade espacial de cache e eliminando a necessidade de concorrência ou comunicação durante o processamento pesado.

Impacto do desbalanceamento: Em algoritmos onde a carga de trabalho por elemento não é constante, como na busca de números primos, onde testar a primalidade de um número grande exige muito mais ciclos de clock do que testar um número pequeno, uma divisão estática simples causa um severo desbalanceamento de carga. As threads encarregadas dos blocos com números maiores demoram muito mais para

concluir, forçando as demais threads (e o núcleo principal) a ficarem ociosas aguardando a finalização da última, o que degrada drasticamente a eficiência do paralelismo.

5.5. Reentrância e Thread-Safety em Bibliotecas (Parte 2 - Opção B)

O problema da função `rand()`: A função tradicional `rand()` mantém um estado interno global (uma semente única oculta) que é alterado a cada chamada. Para evitar que duas threads corrompam esse estado simultaneamente, a implementação padrão na glibc protege a função internamente com um mecanismo de exclusão mútua (*lock*). Isso faz com que todas as threads concorrentes fiquem serializadas ao tentar gerar números aleatórios, anulando completamente qualquer ganho de desempenho proporcionado pelo paralelismo.

Função Reentrante e `rand_r()`: Uma função reentrante é aquela cuja execução pode ser interrompida e reiniciada de forma segura por múltiplos fluxos de execução (*threads*) em simultâneo, sem risco de corromper dados globais ou estáticos, pois todo o seu estado depende exclusivamente de variáveis locais ou de parâmetros passados por ponteiro. A função `rand_r(&seed)` soluciona este problema ao exigir que o programador forneça o endereço de uma variável de semente privada gerenciada de forma independente por cada thread, eliminando qualquer contenção global.

5.6. Escalabilidade e Limitações Físicas de Memória (Parte 2 - Opção C/E)

Comportamento do Speedup: Ao variar de 1 para 2, 4 e 8 threads, os problemas limitados por memória (memory-bound, como a soma de vetores gigantes) não atingem escalabilidade linear.

O teto da largura de banda: A aceleração paralela esbarra rapidamente num limite físico intransponível: a largura de banda da memória RAM (DRAM) e do barramento do sistema. Enquanto as operações aritméticas executadas pela CPU são extremamente rápidas, a taxa com que os dados conseguem transitar entre a memória principal e as caches do processador é limitada pelo hardware. Quando múltiplas threads passam a requisitar dados simultaneamente, o barramento de memória satura rapidamente. Adicionar mais threads além desse ponto não traz melhorias, pois os núcleos passam a ficar ociosos esperando os dados chegarem da RAM.

5.7. O Papel do GIL no CPython (Parte 3)

O *Global Interpreter Lock* (GIL) é um mecanismo de exclusão mútua implementado no interpretador *CPython* para sincronizar a execução de threads, assegurando que apenas uma thread por vez execute o bytecode do Python. Historicamente, essa escolha foi adotada para simplificar o gerenciamento de memória do interpretador, em especial devido ao uso de contagem de referência para a coleta de lixo, que não é segura para threads. Como resultado direto, em tarefas estritamente limitadas por processamento, o GIL impede que múltiplas threads executem operações em paralelo real em processadores multi-core, uma vez que o interpretador serializa a execução, tornando o módulo `threading` ineficaz para obter ganhos de desempenho computacional.

A imagem apresentada na seção 3.3 comprova os resultados da execução dos scripts em Python. Visualiza-se que o tempo de execução da versão sequencial (*seq.py*) e da versão concorrente com threads (*threads.py*) apresentam valores muito próximos, evidenciando a restrição imposta pelo *Global Interpreter Lock* (GIL) do *CPython* em tarefas intensivas de processamento de CPU. Devido a este bloqueio, as threads não conseguem executar em paralelo real nos vários núcleos do processador, o que impede a obtenção de um ganho significativo de desempenho.

5.8. Paralelismo Real em Python (Parte 3)

O módulo padrão multiprocessing contorna a restrição imposta pelo GIL ao instanciar múltiplos processos independentes em vez de threads compartilhadas. Cada processo possui o seu próprio espaço de endereçamento de memória e uma instância exclusiva do interpretador *CPython*, o que anula o efeito do GIL e viabiliza o uso simultâneo de vários núcleos da CPU. Contudo, essa arquitetura apresenta desvantagens e sobrecargas, tais como o custo elevado para criar e destruir processos, a necessidade de serialização de dados utilizando o protocolo pickle para a comunicação entre processos e o maior consumo de memória RAM, visto que os dados precisam ser isolados ou duplicados.

5.9. Limites Teóricos e Lei de Amdahl

A Lei de Amdahl define que a aceleração máxima de um sistema computacional é limitada pela proporção de código que deve ser executada de maneira sequencial. O limite teórico da aceleração é expresso pela fórmula matemática onde o ganho máximo é o inverso da fração sequencial. Ao considerar as frações estritamente sequenciais dos programas desenvolvidos, como a inicialização de estruturas, a alocação dinâmica de memória e as chamadas de criação e finalização de threads, conclui-se que, mesmo com um número infinito de núcleos de processamento, o ganho de desempenho é limitado por essa porção não paralelizável. Logo, se a fração sequencial (*seq*) de uma aplicação representar 5% do tempo total, a aceleração máxima teórica jamais ultrapassará 20 vezes, independentemente da capacidade de hardware disponível.

$$SpeedUp = \frac{1}{(1-f) + \frac{f}{p}}, \text{ mas como no limite teórico: } \lim_{p \rightarrow \infty} \left(\frac{f}{p}\right) = 0$$

$$\text{portanto: } SpeedUp_{MaxTeorico} = \frac{1}{(1-f)} = \frac{1}{seq}$$

6. Seção de Dificuldades Técnicas e Soluções

Na implementação dos algoritmos da Parte 2, ocorreu uma perda de desempenho na execução paralela em relação à versão sequencial. A dificuldade originou-se do uso de um mecanismo de exclusão mútua (mutex) posicionado diretamente dentro do laço principal de cálculo, o que gerou um intenso gargalo entre as threads.

A solução adotada foi reestruturar o código utilizando variáveis acumuladoras locais. Cada thread passou a realizar os cálculos iterativos de forma independente, sem travamentos. O mutex foi então deslocado para fora do laço, sendo acionado apenas uma vez no final da execução de cada thread para agregar o subtotal à variável global. Essa otimização simples eliminou a sobrecarga de sincronização e viabilizou os ganhos reais de aceleração da aplicação.

7. Seção Obrigatória: Declaração e Análise do Uso de IA

7.1. Registro de Uso de Ferramentas

Durante o desenvolvimento deste laboratório e na elaboração do relatório final, utilizou-se o suporte de ferramentas de Inteligência Artificial Generativa, especificamente o Gemini (Google) e o ChatGPT (OpenAI), atuando estritamente como assistentes auxiliares e não como autores principais do trabalho.

O Gemini foi empregado principalmente como um revisor de texto e parceiro de formatação. Sua função foi lapidar os rascunhos redigidos manualmente, conferindo maior formalismo acadêmico, coesão estrutural e clareza às frases das seções teóricas, além de sugerir mudanças em possíveis erros conceituais. Além disso, o Gemini foi consultado para auxiliar no entendimento do gargalo de desempenho relatado na Seção 6. Por sua vez, o ChatGPT foi acionado de forma pontual para a geração de um script utilitário em Python (cerca de 30 linhas). O objetivo desse código era exclusivamente automatizar a leitura de uma tabela Excel preenchida integralmente com dados empíricos de tempo coletados nos códigos e plotar os gráficos de Speedup e Eficiência em formato PNG para inclusão no documento.

7.2. Listagem das Principais Solicitações (Prompts)

Abaixo estão listados os principais comandos formulados e os contextos em que foram aplicados:

Prompt 1 (Gemini - Lapidação de Texto): "Aqui está um rascunho que escrevi sobre a introdução ao Pthreads e os problemas de condição de corrida. Reescreva este texto focando em um tom acadêmico e em terceira pessoa, formalizando a linguagem e garantindo clareza técnica, mas sem alterar os conceitos que eu descrevi. Se houve qualquer erro conceitual, me sinalize e me dê fontes para buscar revisar o conceito"

Prompt 2 (Gemini - Depuração de Conceito - Seção 6): "Estou implementando a Soma de Vetores na Parte 2 e meu desempenho multithread está pior que o sequencial. Eu coloquei um mutex de controle bem restrito dentro do laço principal. Explique por que isso gera gargalo e como posso reestruturar o código de forma mais eficiente."

Prompt 3 (ChatGPT - Geração de Script de Plotagem): "Crie um código Python simples, usando Pandas e Matplotlib. Ele deve ler um arquivo 'tabelas_tempos.xlsx' que eu gerei com alguns tempos de execução, calcular o Speedup e a Eficiência, e salvar em gráficos PNG. Não precisa gerar os dados, apenas plotar o que está na planilha."

7.3. Análise Crítica Obrigatória

A experiência com as ferramentas de IA demonstrou que elas são recursos valiosos para ganho de produtividade, desde que supervisionadas rigorosamente. No aspecto de programação utilitária (o script de geração de gráficos no ChatGPT), a ferramenta foi cirúrgica: a resposta foi precisa e o código funcionou na primeira tentativa, eliminando o tempo que seria gasto consultando a documentação do Matplotlib.

No entanto, no aspecto conceitual de paralelismo, notou-se a necessidade de senso crítico. Ao investigar o problema de contenção do mutex relatado na Seção 6, o Gemini sugeriu inicialmente soluções altamente complexas para mitigar o gargalo, como a implementação de Read-Write Locks (*pthread_rwlock_t*), o que representava um nível de complexidade alto. Foi necessária a intervenção e o discernimento humano para descartar essa sugestão superdimensionada e guiar a IA para uma abordagem arquitetural mais simples.

Essa dinâmica evidenciou que a aprendizagem real adquirida na prática de laboratório não pode ser terceirizada. O uso ético e transparente dessas ferramentas permitiu focar no que realmente importava: a análise arquitetural e matemática do paralelismo, automatizando apenas as etapas mecânicas de formatação de texto e renderização gráfica.

8. Conclusão e Referências

Esse laboratório serviu para evidenciar ainda mais que a programação paralela em memória compartilhada reduz o tempo de execução de tarefas complexas, mas exige rigor no controle de acesso aos recursos compartilhados para evitar inconsistências e condições de corrida. O uso inadequado de mecanismos de sincronização, como o busy-waiting, desperdiça ciclos de processamento, enquanto travas de granularidade excessiva geram gargalos severos de contenção.

A escolha correta das primitivas de sincronização, sejam Mutexes, Read-Write Locks ou Semáforos, deve ser ponderada de acordo com a natureza da aplicação e a frequência de operações de leitura versus escrita. Por fim, restrições arquiteturais em ambientes de alto nível reforçam que o domínio dos conceitos fundamentais de concorrência é indispensável para extrair o máximo desempenho de arquiteturas multi-core.

Referências Bibliográficas:

PACHECO, P. *An Introduction to Parallel Programming*. New York: Elsevier, 2011

PYTHON SOFTWARE FOUNDATION. *Global Interpreter Lock (GIL)*. Python Glossary, 2024. Disponível em: docs.python.org/3/glossary.html. Acesso em: 20 set. 2026.